

Creating the PAYE panel

Contents

Contents.....	1
1. High-level summary of data creation.....	2
2. Source data structure	5
3. Occupational pensions.....	6
4. Creating useful variables.....	7
5. Allocating weeks and months in the PAYE data.....	8
Stage 1: remove duplicates	8
Stage 2: Allocate extra days to weeks 1-52	8
Stage 3 Allocating months and weeks to days	9
Creating monthly/weekly data in pictures.....	10
Payslip data – overall plan	10
Cleaning and preparing weekly data.....	11
Cleaning and preparing monthly data	11
Combining monthly and weekly data to make weekly pay	11
Combining monthly and weekly pay to make monthly pay	12
Appendix 1: detailed code review	13
File: HMRC_extract_data	13
File: HMRC_create_PAYE_panel.....	13
File: Create_derived_vars.....	22

1. High-level summary of data creation

This document describes how the PAYE panel is created. The steps are as follows:

1. Clean up source file, removing duplicate payslip entries
2. Create ASHE lookup of age/sex by year and piden to be added to HMRC data
3. Create markers for occupational pensions and their schemes
4. Create clean weekly and monthly datasets
 - a. Add age and sex
 - b. Separate weekly, monthly and unknown payslips into 3 files
 - c. Make adjustments for multi-week/multi-month payments (4 weeks or 3 months)
 - d. keep and weekly/monthly 'clean' files which just have those payslips in
5. Combine new weekly and monthly files into one new file, the 'weekly panel':
 - a. Construct weekly data from monthly files
 - b. Include weekly data from weekly files
6. Repeat to create a 'monthly panel'
7. Create markers eg of employment activity from the payslip data
8. Extract useful additional information from the Employment and EoY files
9. Create person-level summary, by hmrc_id
10. Aggregate by person by year (the 'ASHE supplement')

This generates five files

File name	Contents	Use
paye_weekly	Pay slip records for those paid weekly or in multiples of weeks, augmented by age and sex from ASHE	Analysis of size and volatility of weekly pay
paye_monthly	Pay slip records for those paid monthly or in multiples of months, augmented by age and sex from ASHE	Analysis of size and volatility of monthly pay
paye_weekly_panel	All weekly and monthly pay slip data with months allocated to weeks, with additional constructed data on employment records, augmented with age and sex markers from ASHE	Analysis of pay, labour market transitions (including employer transitions using the PAYE index scheme as a proxy for employer)
paye_monthly_panel	All weekly and monthly pay slip data with weeks allocated to months, with additional constructed data on employment records, augmented with age and sex markers from ASHE	Analysis of pay, labour market transitions
paye_ashe_supplement	Pay slip data for an individual in a year, summed across all jobs, including information on the number of jobs, proportion of time in paid employment, labour market activity at the ASHE survey date	Analysis of the ASHE data (including sampling issues) using information on within-year activities

The first four are for analysing pay throughout the year. The last is to be added to ASHE to throw light on the ASHE data, particularly the information about what happens in the rest of the year.

There are a very small number of payslips (around 1,300 out of 38m) which do not have either a weekly or monthly period number. At present the payslip file supplied contains no useful information for these records (all non-administrative fields are zero or missing), so no further actions has been taken with it and the records dropped.

A high-level summary of the files and their characteristics can be found in the *ASHE-WED-HMRC Quick Guide for Users* on the WED website¹. This also holds the detailed metadata in an Excel spreadsheet, and *Guide to linking ASHE and HMRC data* which explains how the various personal identifiers link together.

The detailed code description can be found below.

¹ <https://www.wagedynamics.com/hmrc/>

Data quality checks

We list, in the following sections, tests that have been carried out on the data to get the data into a fit state for its initial release. There are a significant number of quality checks still to be carried out. Tests already identified by the WED team include

- Interpretation and reliability of variables: whether zero-payslips are indicative of zero-hours working; interpretation of End-of-year data; hours working data;
- Consistency within HMRC payslip data: whether year-to-date (YTD) variables can be used to reliably indicate missing payslips; whether individuals flip between weekly and monthly pay on the same job; multiple job holding
- Consistency between HMRC files: between start and end dates in employment file and as observed in the payslip data; between pension status in the End of Year file and occupational pensions observed in the pay data (very little correlation apparent); between employment and recorded employment in the SA sdata
- Consistency between ASHE data and HMRC data: number of employers, length of employment, total wages

In addition, we need to understand better the interplay between occupational pensions and employment, to refine the definition of 'working'.

These will be explored with the aim of producing a new QA report in October 2024. Researchers starting to work on this data are encouraged to share queries and findings with the WED team for further dissemination.

2. Source data structure

The input dataset 'payslips' contains all payslips for all different payment periods, plus 'weekly_period_number' and 'monthly_period_number' to indicate when the payment refers to:

	Weekly		Monthly		Other	
	W1	Weekly	M1	Monthly	IO	Irregular (one-off)
	W2	Bi-weekly	M3	Three-monthly	IR	Irregular
	W4	Four-weekly	M6	Six-monthly		Missing
			MA	Twelve-monthly		
Weekly period	Valid		Missing		Missing	
Monthly period	Missing		Valid		Missing	

In the current dataset, the 'other' fields have no meaningful information and so have been ignored.

We have used the validity of the monthly/weekly period number as the indicator of weekly or monthly pay – they are generally consistent with the pay frequency variable. There are a very small number of inconsistencies, with the exception of the 'irregular' payments: see the table below.

[table to be extracted from SRS]

For irregular, if this is genuinely 'irregular' then appearing as a one-off weekly or monthly payment seems like a fair approximation of 'irregular'. If this is combined with other weekly or monthly pay which is not the same type, then this will be picked up as a change in payment arrangements, which seems sensible.

For the small numbers of individuals we have a weekly payment period but appear to have frequency "M1", we suspect this might be new starts or other job changes. Again, this will be picked up in the change variable if the M1 is correct but the weekly number is not, or vice-versa.

When expanding from a single multi-period payment to the correct number of separate payment (eg single M3 payment => 3 monthly payments) we assume that the payment is made in the last of the periods (ie month 3 in this example).

3. Occupational pensions

Roughly 25% of the observations are identified as occupational pension payments, rather than payments for employment, by the source data flag **occ_pen**. HMRC advises that this is generally reliable but not foolproof. In reviewing the data, it is clear that there are individuals who get mostly occupational pensions but occasional employment payments, or vice versa.

The variable **index_scheme** distinguishes between PAYE schemes (companies may have more than one scheme)². There are around 300,000 PAYE schemes registered in the PAYE data. It appears that some schemes are only used to pay wages (the overwhelming majority), some are used only to pay occupational pensions about 1%), and a very small number appear to do both. Following advice of HMRC and ONS, we use this information to adjust the raw **occ_pen** variable, by the following procedure (see Appendix for details):

- Identify each PAYE scheme as ‘employees’, ‘occupational pensions’, or ‘mixed’
- When preparing the payslip data, check to see if all payslips consistent
- If there are apparent inconsistencies, update by comparison with the scheme definition

The code also identifies each individual (per year, per PAYE scheme) as a ‘pensioner’ or not, based on whether they are only appear to be receiving occupational pensions.

Note that ONS uses slightly different criteria to the WED team to adjust for occupational pensions. Some of this uses variables which are not currently available to the WED team. One variable which is available is the **hours_worked_category** variable. This is banded (ie it does not represent actual hours). According to the HMRC website <https://www.gov.uk/guidance/what-payroll-information-to-report-to-hmrc> this should be:

Code	Hours worked
A	<16
B	16-23.99
C	24-29.99
D	30+
E	Workplace pension, annuity, or no regular payment

The variable **hours_worked_category** takes the values [20, 21, 22, 23, 24], which we understand relates to A-E above. ONS considers that **hours_worked_category=24** should also be taken into account, as supporting information (in combination with other data) but not definitive. However, an examination shows that there appear to be many more observations in category 24 than would be expected; it is usually the largest category of hours reported, even when taking out the occupational pensions. As it is not clear to us how this provides strong evidence of an occupational pension, we have therefore only taken account of the PAYE scheme status and not hours. This does also suggest **hours_worked_category** should be treated with caution.

² **Index_scheme** is used to identify employers because the variable **index_employment**, if interpreted as different jobs, has an implausible distribution of jobs. A third employment reference **emp_id** appears to be an ONS-added reference which does not clearly link to other data. ONS has committed to deliver the IDBR enterprise reference number **ent_ref** but there is no delivery date for this as of June 2024.

4. Creating useful variables

In this section, we associate working with getting a payslip. So, no pay means not working. We treat all data identified as paid from a PAYE index scheme as indicating a separate employer payment ['job'], which could be pension or employment. When providing the year-end summary, we distinguish between employment and occupational pensions.

These are the additional variables associated with the **paye_weekly_panel** and **paye_monthly_panel** which we then aggregate over the year and jobs to get the **ashe_supplement**.

1. First, add the survey ref week and month number for each tax year to the data
2. For each person and tax year and employment (in the HMRC data, to keep it easy, and then sum by person and year), can we extract (and create as a dataset indexed by year and piden):

to be used for sampling error checks:

- Whether working on the ASHE survey ref date
- Whether working at all in April of the tax year
- Whether working at all in March of the previous tax year

to be used for ASHE analysis:

- Weekly pay in the survey ref week
- Number of different jobs in previous year
- Number of jobs at the reference point
- Number of different employers in previous year
- Whether working at all in previous year
- Whether working at all in current year
- % of time (number of working weeks/52) in previous year
- % of time (number of working weeks/52) in current year
- Average weekly pay in previous year when working
- Average weekly pay in current year when working
- St deviations of weekly pay in previous/current
- Whether weekly or monthly paid or both

We also add the cardinal value 'age' and the binary variable 'male' to the monthly and weekly payslips. This is done at the first stage of cleaning.

5. Allocating weeks and months in the PAYE data

The PAYE payslip data contains payslips, for each job in each period where payment is made. Each payslip is assigned a pay period number: weekly from 1-56, monthly from 1-12, and one-off payments.

To make a usable panel from the PAYE data, we need to take this data and convert it all into either weekly, monthly or yearly payments, but the HMRC payment periods do not align. We also need to remove duplicates. Finally we need to address payments that don't come in regular times.

We assume that you can have weekly, monthly and one-off payments within a year, but not at the same time.

Test: Check whether monthly and weekly payments can occur at the same time

Result: As measured by 'weekly period number' or 'monthly period number', a payslip must be one or the other but not both (a very small number of 'other' payments are neither). Hence this is used to split data into weekly and monthly pay. The tiny number with both weekly and monthly period number blank have neither pay frequency or other useful information, and so are dropped.

Stage 1: remove duplicates

There are many duplicate payslip records for each time period. We understand that this represents corrections to the data by the employer: the very high number of duplicates is caused by employers sending updated records for all their employees when a change has to be made, rather than the corrected payslip for the employee affected.

The one-off payments are not affected by this.

Solution

We delete all but the most recent payslip for that period using the reference number. Note that ONS does a similar thing but deleting by 'last updated' date and then reference number if there are duplicate entries on the last updated day. If index numbers are sequential (as the ONS solution implies) then our solution should have the same effect but be more efficient. The ONS team thinks the two approaches are the same.

Test: Before deleting, check that most of the payslips in the same period of the same job have the same values

Result: [ONS has tested this and is satisfied with the result]

Stage 2: Allocate extra days to weeks 1-52

There are 52 weeks in a year plus one or two extra days, depending on the leap year. Week 1 of the tax year always starts on 6th April in the calendar year and week 52 ends on 3rd or 4th April the following calendar year (depending on whether a leap year).

The payslips file also include entries for weeks 53, 54 and 56. These are additional payments to cover the extra day(s), and the code is used for those paid weekly, bi-weekly and four-weekly, respectively.

Solution

We allocate the week 53-56 payments as follows:

- For week 53 (weekly paid), add to week 52
- For week 54 (bi-weekly paid), share amongst week 51 and 52
- For week 56 (four-weekly paid), share amongst weeks 49-52

Test: Check if the 1,2, 4 week pay period is consistent with the employment file

Result: Generally yes. A small number of payslips (<<0.01%) with a nonzero weekly pay period are listed as having monthly pay frequency, and vice-versa.

Stage 3 Allocating months and weeks to days

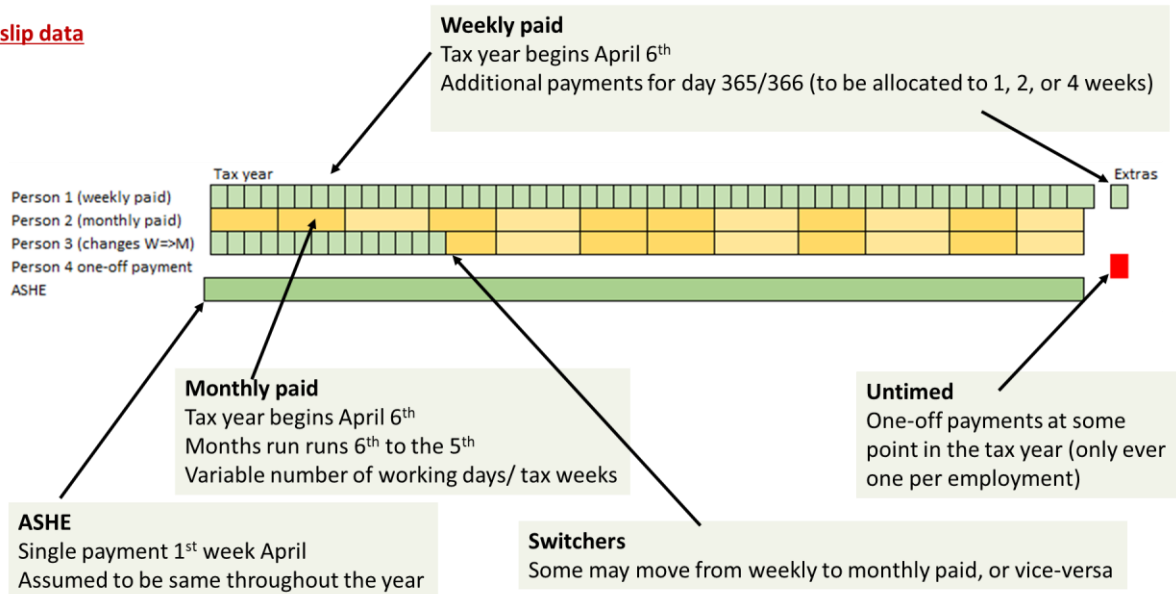
1. Generate tax month as month/year pair and number of days in the month (1:1)
2. Generate tax_week as week/month/year trio with number of days in the week in each month (1:m – one or two lines per tax week)
 - a. Note we also need to retain the extra 1/2 days for the 365th/366th day of the year when constructing monthly wages (but **not** for the weekly wages – see code)
3. Convert into day rates:
 - For months, merge with monthly file to get actual days and then divide – save as payslips_m_daily
 - For weeks, divide by 7 – save as payslips_w_daily
4. Calculate weekly pay
 - For monthly paid:
 - Load up monthly day rates and expand to the weeks that that month covers
 - Multiply monthly day rate by the number of days in that week eg
 - week 5 has 2 days in month 1 and 5 days in month 2 =>
 - Week 4 wage = 7 x month1 rate
 - Week 5 wage = 2 x month 1 rate + 5 x month 2 rate
 - Week 6 = 7 x month 2 rate
 - Then sum by week
 - Then add on the weekly paid from paye_weekly
 - Combine the two and sum as weekly_pay_combined
 - Create additional variables
5. Calculate monthly pay:
 - For weekly paid:
 - Get the weekly_paid day rate file
 - Merge with tax_weeks to get all the months that that tax_week appears in
 - Sum day rates for all the weeks for each day in that month eg
 - week 5 has 2 days in month 1 and 5 days in month 2 =>
 - month 1 salary = week1x7 + week2x7 + week3x7 + week4x7 + week5x2
 - month 2 salary = week5x5 + week6x7 etc etc
 - collapse by month
 - For monthly paid
 - Merge the paye_monthly onto this file

- Create additional variables
- Save as `paye_monthly_panel`

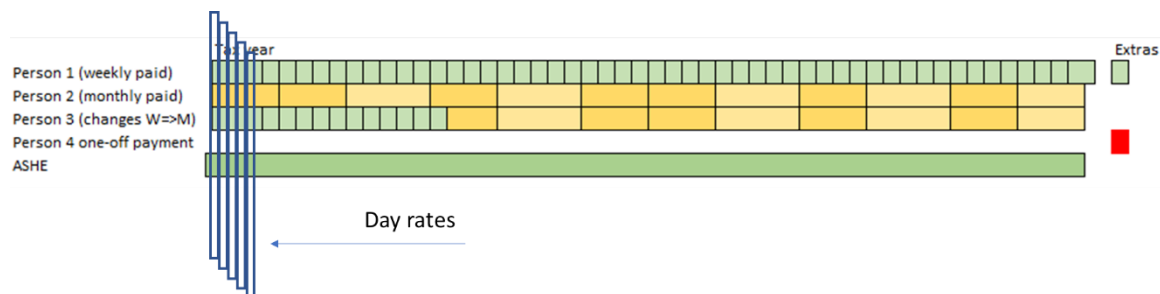
Creating monthly/weekly data in pictures

The input files are structured as follows:

Payslip data

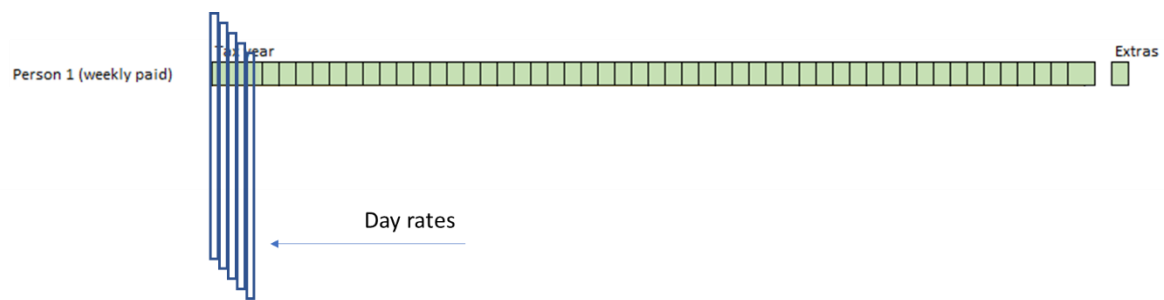


Payslip data – overall plan



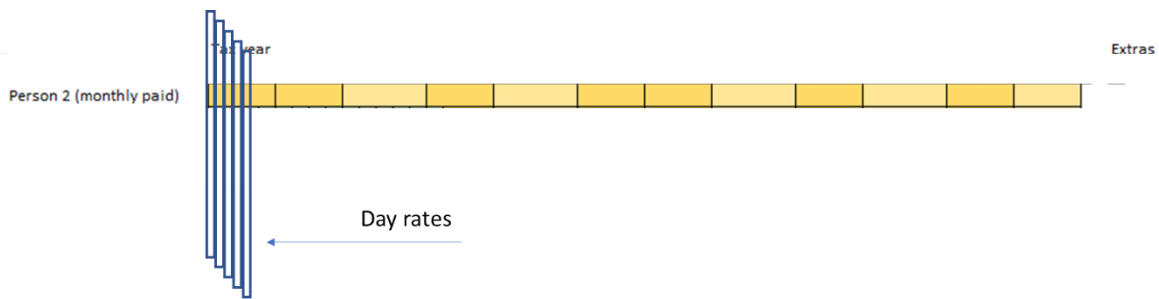
1. Convert all multi-period pay (3 month, 4 week etc) to single periods; save as 'clean' monthly/weekly pay
2. Convert all data to daily pay- save as monthly/weekly day rates
3. Reconstruct weekly and monthly pay by adding up relevant day rates
4. Construct job history (current job, n jobs, inactivity)
5. Calculate total weekly and annual income per person (for comparison with ASHE)
6. Calculate annual activity measures

Cleaning and preparing weekly data



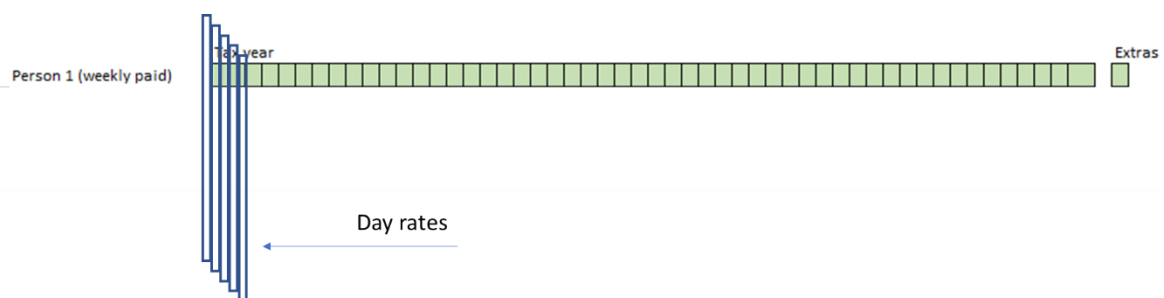
1. Separate off the weekly data (including switchers)
2. Expand 2-week and 4-week pay periods into single week payments
3. Allocate extra days (day 365/366) as appropriate, depending on whether 1-, 2- or 4-week pay period
4. Add a marker for tax month
5. Save as 'clean' weekly payments
6. Divide by 7 to get daily payments for the weekly-paid, and save

Cleaning and preparing monthly data



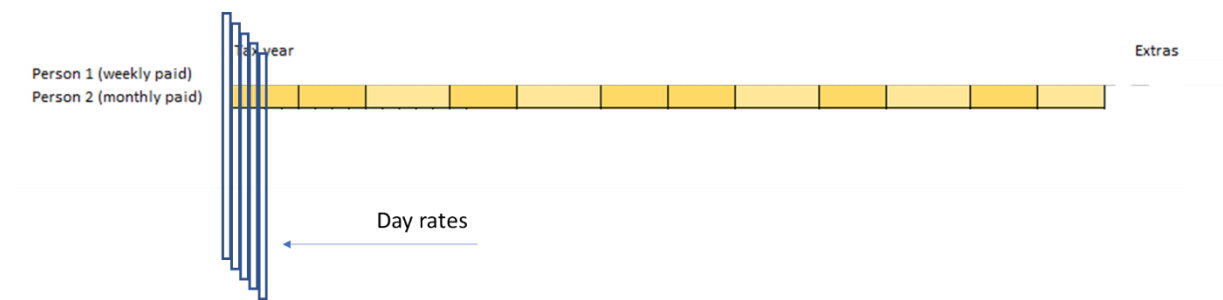
1. Separate off the monthly data (including switchers)
2. Expand 3-, 6- and 12-month pay periods into single month payments
3. Add the number of days in the month
4. Save as 'clean' monthly payments
5. Divide by appropriate number of days to get daily payments for the monthly-paid, and save

Combining monthly and weekly data to make weekly pay



1. Expand months to the number of (partial) weeks in that month, noting the days in the week in that month
2. Multiply monthly day rates with number of days in the month
3. Sum by week (some weeks will span two months)
4. Mark as 'monthly paid'
5. Add on the original weekly data and mark this as 'not monthly paid'
6. Save as 'weekly panel'

Combining monthly and weekly pay to make monthly pay



1. Combine week numbers with number of days in the month
2. Multiply by the number of days of that week in that month by the daily rate for the week
3. Sum by month
4. Mark as 'not monthly paid'
5. Add on the original monthly data and mark this as 'monthly paid'
6. Save as 'monthly panel'

Appendix 1: detailed code review

File: HMRC_extract_data

tbd

File: HMRC_create_PAYE_panel

Macros:

- Load up relevant macros for input and output files, variable lists, which bits of the program to run
- There are 5 program control macros; all but the first are very slow:
 - *create_ashe_lookups*: prepare the ASHE age/sex variables
 - *do_initial_splits*: calculate occ peions adjustments, separate payslips file into weekly, monthly, other files
 - *construct_clean*: generate clean monthly/weekly/other files and the daily rates
 - *construct_combinedw*: generate weekly pay files; needs weekly clean files and monthly day rates from previous step to run
 - *construct_combinedm*: generate weekly pay files; needs monthly clean files and weekly day rates from previous step to run

Note: all the analysis below is done by person, tax year and job, until we start combining at the person level.

We initially developed this code looking at employment_index as the guide to employment. However, this creates nonsense numbers – many people with hundreds of jobs in a year. We don't have employment_seq_number in the current drop, so we are using index_scheme instead. This gives plausible counts for employment (50% have one job in a year, 20% have two, only 0.5% have 12 or more) but the tails are still too heavy, so we suspect some administrative error in the data.

There is also an element of under-estimation as using the PAYE scheme means that someone who works eg Tesco, then Asda, then Tesco again will only be noted as 2 jobs, but we think is likely to be a minor error.

If we get employment_sequence_number reinstated, we will run again using that as the job marker and see what changes.

Stage 0: separate create ASHE lookups	
Input files	ASHE panel (all payslips) <i>[temp folder]</i>
Output files	ashe_age_sex_lookup <i>[temp folder]</i>
Structure of the output file	One observation per individual, indexed by piden
existing vars in the output files	sex, with inconsistencies removed
New variables created in the output file	birth_year, with inconsistencies removed

Activity:

- Load up the ASHE panel, keeping only year, age and sex
- Create birth_date as year-age
- Take the model value of age and sex, reduce to one observation per individual, and save

Stage 1: separate payslip file into 3 parts	
Input files	payslips_file (all payslips) <i>[temp folder]</i>
Output files	paye_pensioner_status, paye_scheme_pension_status, payslips_w, payslips_m, payslips_o <i>[temp folder]</i>
Structure of the output file	Payslips data, by week, month or other pay date
existing vars in the output files	'save_vars'
New variables created in the output file	Age and gender from ASHE

Activity:

- Load up the payslips file
- Merge with the employment data (needed for PAYE scheme and pay frequency)
- Identify PAYE scheme type and save (index by PAYE scheme)
 - Occupation pension if >99% of payments are occ pensions
 - Employment if <1% are occ pensions
 - Mixed if anything else
- Identify pensioner type: by person, tax year and PAYE scheme:
 - Set this job as a 'pension' if >79% of observations have occ_pen==1
 - Count the number of 'employments' and 'pensions' and replace the data for the person by this info
 - Set this person as 'pensioner' if they have no paid employments, only pensions
 - Save (indexed by person, tax year)
- Generate some descriptive stats to show that each obs has one, and only one, of [weekly, monthly, other] pay period ref
- Save into 'weekly' 'monthly' and 'other' files, depending on pay period

Stage 2 step 1: construct clean monthly pay from monthly files and monthly day rates	
Input files	payslips_m (monthly payslips) <i>[temp folder]</i>
Output files	paye_monthly <i>[clean folder]</i>
Structure of the output file	Payslips data, by person, year, month, job; 3- and 6- month payment dates extrapolated to single months (for both scale and scale-YTD variables); indexed by hmrc_id, tax year and job
existing vars in the output files	'save_vars'
New variables created in the output file	tax_month : tax month number for payments received monthly_pay: pay allocated to months (if 3- or 6-month) pay_ytd: cumulative pay year-to-date

Activity:

- Load up the payslips file
- Compare taxable_pay_ytd (accumulated pay YTD) and pyt_taxable_pay_in_period (pay YTD as reported on payslip) (Results: mostly the same but about 1/8 different)
- Monthly pay can be every 1, 3, 6 or 12 months; so generate monthly equivalents (eg a single £6,000 payment for 3-monthly paid period in month 6 becomes 3 monthly payments of £2,000 in months 4, 5, 6) – see example below

Paid every month

HMRC_ID	tax_year	month	taxable_pay_in_p eriod	taxable_pay_ytd
123456	2015	1	2000	2000
123456	2015	2	2000	4000
123456	2015	3	2000	6000
123456	2015	4	3000	9000



HMRC_ID	tax_year	m2	monthly_pay	pay_ytd
123456	2015	1	2000	2000
123456	2015	2	2000	4000
123456	2015	3	2000	6000
123456	2015	4	3000	9000

Paid every 3 months

HMRC_ID	tax_year	month	taxable_pay_in_p eriod	taxable_pay_ytd
678910	2015	3	6000	6000
678910	2015	6	7500	13500



HMRC_ID	tax_year	m2	monthly_pay	pay_ytd
678910	2015	1	2000	0
678910	2015	2	2000	0
678910	2015	3	2000	6000
678910	2015	4	2500	6000
678910	2015	5	2500	6000
678910	2015	6	2500	13500

Stage 2 step 2: construct monthly day rates	
Input files	paye_monthly <i>[clean folder]</i>
Output files	payslips_m_daily <i>[temp folder]</i>
Structure of the output file	Monthly payslip data converted into a day rate
existing vars in the output files	'scale_vars' (ie data on wages paid in that month), converted to a daily rate
New variables created in the output file	days_in_month

Activity:

- Load up the monthly payslips file
- Merge with **tax_months** on tax_month and tax_year (both to allow for leap years) to the days-per month variable
- Divide the scale variables by the days in the month (ytd variables left as before)
- Save as payslips_m_daily

Stage 2 step 3: construct clean weekly pay from weekly files	
Input files	payslips_w (weekly payslips) <i>[temp folder]</i>
Output files	paye_weekly_clean <i>[clean folder]</i>
Structure of the output file	Payslips data, by person, year, week, job; extra days allocated to last 1/2/4 months as relevant; 2- and 4-week payment dates extrapolated to single weeks
existing vars in the output files	'save_vars'
New variables created in the output file	tax_week: tax week number for payments received

Activity:

- Load up the payslips file
- Create tax_w_derived (weekly payment period number) as 1-52
- Create extra_pay by job, covering the 365th day
- Add week 53 into week 52 (weekly paid), week 54 into weeks 51-52 (bi-weekly paid) and week 56 to weeks 49-52 (four-weekly paid)
- Create a variable n_week to represent whether 1, 2 or 4 week payment periods, and use this to expand the payslips to complete weeks 1-52
- Create a new variable tax_week to represent the new weekly period numbers
- Create new pay variables, allowing for expanded values

Paid every week

HMRC_ID	tax_year	week	taxable_pay_in_period	taxable_pay_ytd
123456	2015	50	500	25000
123456	2015	51	500	25500
123456	2015	52	500	26000
123456	2015	53	100	26100



HMRC_ID	tax_year	w2	weekly_pay	pay_ytd
123456	2015	50	500	25000
123456	2015	51	500	25500
123456	2015	52	600	26100

Paid every 4 weeks

HMRC_ID	tax_year	month	taxable_pay_in_period	taxable_pay_ytd
678910	2015	48	2000	24000
678910	2015	52	2000	26000
678910	2015	56	400	26400



HMRC_ID	tax_year	w2	weekly_pay	pay_ytd
678910	2015	45	500	22000
678910	2015	46	500	22000
678910	2015	47	500	22000
678910	2015	48	500	24000
678910	2015	49	600	24000
678910	2015	50	600	24000
678910	2015	51	600	24000
678910	2015	52	600	26400

Stage 2 step 4: construct weekly day rates	
Input files	paye_weekly <i>[clean folder]</i>
Output files	payslips_w_daily <i>[temp folder]</i>
Structure of the output file	Weekly payslip data converted into a day rate
existing vars in the output files	'scale_vars' (ie data on wages paid in that month), converted to a daily rate
New variables created in the output file	

Activity:

- Load up the weekly payslips file
- Divide the scale variables by 7 (ytd variables left as before)
- Save as payslips_w_daily

Note: this assumes that the year only has 364 days – the extra day(s) of wage is absorbed into week 52. But we will need to add on the extra days when we convert to months, otherwise month 12 will be short.

Stage 3 step 1-2: merge clean monthly and weekly files to create weekly payslips data and annual pay (bottom up)	
Input files	payslips_m_daily <i>[temp folder]</i> paye_weekly <i>[clean folder]</i> payslips_w_daily <i>[temp folder]</i> paye_monthly <i>[clean folder]</i> [Note the 'other' data (non-weekly, non-monthly) is not recombined as it has no valid data fields]
Output files	annual_pay_bottom_up <i>[temp folder]</i> annual_pay_top_down <i>[temp folder]</i> paye_weekly_panel <i>[clean folder]</i> paye_monthly_panel <i>[clean folder]</i>
Structure of the output file	Weekly and monthly payslips data; where there is overlap (change w↔m within a week/month), added together; YTD information (pay and student loans) not retained as unclear meaning Annual summary, from adding up week and months
existing vars in the output files	'save_vars' excluding YTD variables
New variables created in the output file	monthly_paid: binary for if monthly paid or not mixed: employee had both monthly and weekly pay in that period

Activity

- For the weekly file:
 - Load up monthly daily pay rates
 - Merge with tax_weeks to work out how many days need to be allocated to which weeks
 - Calculate the total pay in a week by the number of days in that week allocated to that month (use the extra days at the year end), and multiplying by the appropriate month rate
 - Calculate YTD variables as the monthly value if this is the last week in the month, otherwise as the previous week's value
 - Append weekly clean files
 - Where individuals have both weekly and monthly pay, sum over weeks
 - Save as paye_weekly_panel
 - Sum up to create annual wages for checking
 - Save as payslips_annual_bottom_up
- For the monthly file:
 - Load up weekly daily pay rates

- Merge with tax_weeks to work out how many week days need to be allocated to which month
- Calculate YTD variables as the maximum weekly value in that month
- Calculate the total pay in a week by the number of days in that week allocated to that month (max 7 days – don't use day 365/366), and multiplying by the appropriate week rate
- Append monthly clean files
- Where individuals have both weekly and monthly pay, sum over months
- Save as paye_monthly_panel
- Sum up to create annual wages for checking
- Save as payslips_annual_top_down
- Compare annual pay from the two methods to make sure they give the same outcome (they do – allowing for rounding errors)

Examples:

Monthly to weekly pay

tax month	monthly pay	days in month	daily pay	pay YTD
1 Apr	2000	30	66.67	2000
2 May	2000	31	64.51	4000
3 Jun	2000	30	66.67	6000
4 Jul	3000	31	96.77	9000



tax month	tax week	Days in week in month	pay in week in month	total weekly pay	pay YTD from file	Actual pay YTD
1	1	7	466.67	466.67	0	
1	2	7	466.67	466.67	0	
1	3	7	466.67	466.67	0	
1	4	7	466.67	466.67	0	
1	5	2	133.33	455.91	2000	2323
2	5	5	322.58	(combined)		
2	6	7	451.61	451.61	2000	
2	7	7	451.61	451.61	2000	
2	8	7	451.61	451.61	2000	
2	9	5	451.61	455.91	4000	4144
3	9	2	66.67	(combined)		
3	10	7	466.67	466.67	4000	

Weekly to monthly pay

tax week	Days in week (month)	pay in week in month	daily pay	pay YTD from file
1	7 (1)	700	100	700
2	7 (1)	700	100	1400
3	7 (1)	700	100	2100
4	7 (1)	700	100	2800
5	2 (1) 5 (2)	700	100	3500
6	7 (2)	700	100	4200
7	7 (2)	700	100	4900
8	7 (2)	1050	150	5950
9	5 (2) 2 (3)	1050	150	7000
10	7 (3)	1050	150	8050



tax month	Calculation	Monthly pay	Reported pay YTD	Actual pay YTD
1 Apr	$(4 \times 7 \times 100) + (2 \times 100)$	3000	2800	3000
2 May	$(5 \times 100) + (2 \times 7 \times 100) + (1 \times 7 \times 150) + (5 \times 150)$	3700	5950	6700

note: week 52 will have 8 or 9 days in month 12

For a numerical example, see *example of calculating weekly-monthly wages from monthly-weekly pay.xlsx* in the support folder. Note: this uses actual data, so don't extract from the SRS.

File: Create_derived_vars

This file has three stages (analysis of payslips, EoY data, employment data), controlled by these macros

- *create_vars_from_payslip*: extract the payslip variables into temporary files
- *combine_payslip*: bring all the temp files together (note: two macro because in development we only want to rerun some of the temp file creation, as they take a long time)
- *create_vars_from_eoy*: extract EoY data into temp file and combine with previous step
- *create_vars_from_employment*: extract employment data into tempfile and combine with previous step
- *create_complement*: bring all the data together
- *tidy_up*: delete temporary files – done as a separate step so we can check through development

Stage 1 Extract data from the payslip file, mostly the weekly panel into a series of temporary files	
Input files	payslips [<i>temp folder</i>] paye_weekly_panel [<i>clean folder</i>] pidens [<i>temp folder</i>]
Output files	payslip_temp_combifile [<i>temp folder</i>]
Structure of the output file	Several temporary files indexed by HMRC_id and tax year, each containing one or two pieces of information
existing vars in the output files	hmrc_id, tax_year
New variables created in the output file	variables as described below

This file goes through and generates a series of temporary files, indexed by hmrc_id and tax_year, to be merged into the ASHE.

The following tempfiles are all created (the macro names are temp_ followed by the number below). Note that we identify 'working' with 'receiving a payslip'.

The odd numbers are for the current year, and are matched on tax_year. The even numbers are for the previous tax year, and are matched on tax_year-1.

file	Variable(s)	Meaning/derivation
1	working_survey_ref	Whether the person was working at the time of the ASHE survey date (ie on the data collection date) across all jobs Whether working_survey_ref = 1 iff working during the survey week (weekly paid) or survey month (monthly paid)
2	working_mar_prev_y prev_tax_year	Whether the person was working during the March prior to the ASHE survey (ie during the sample selection time) across all jobs Working_mar_prev_y = 1 iff working during the survey week (weekly paid) or survey month (monthly paid) prev_tax_year renamed from tax_year
3	weekly_pay_survey_ref	Mean (across all jobs held) of weekly pay in the survey week
4	num_job_prev_y prev_tax_year	N jobs in the previous year (whether concurrent or not) num_job_prev_y = sum of different PAYE schemes during the previous year prev_tax_year renamed from tax_year
5	num_job_survey_ref	N jobs at the survey reference point num_job_survey_ref = sum of different PAYE schemes at the point where w2 == survey reference week

6a	num_employer_prev_y_ent prev_tax_year	Number of employers in the previous year As determined by the 'enterprise'-type variable
6b	num_employer_prev_y_entgrp prev_tax_year	Number of employers in the previous year As determined by the 'enterprise group'-type variable
7	working_in_y	If working at all in the year
8	working_prev_y prev_tax_year	If working at all in the previous year
9, 10	avg_weekly_inc total_weekly_inc std_avg_weekly_inc std_total_weekly_inc p_working_weeks (prev_tax_year)	Amount of time working last year, and mean and s.d. of weekly wages in current year Same but for previous year, suffixed 'prev_y' avg_weekly_inc = mean pay across all jobs in a week, averaged across the year total_weekly_inc = total pay across all jobs, averaged across the year std_avg_weekly_inc = s.d. of pay mean weekly pay std_total_weekly_inc = s.d. of total weekly pay p_working_weeks = no of working weeks/52 (NB this will overestimate for monthly paid)
11	count_taxcode	Number of separate tax codes associated with that person during the year

These files are then combined into one file ready to be merged with the next set. If 'tidy_up==1, the intermediate files are deleted.

Stage 2 Extract data from the EoY file	
Input files	paye_eoy [temp folder] pidens [temp folder]
Output files	eoy_temp_combifile [temp folder]
Structure of the output file	Temporary files indexed by HMRC_id and tax year
existing vars in the output files	all variables in the EoY file
New variables created in the output file	None

This code collapses on the EoY data from multiple jobs, to give a single record for an individual and a tax year.

Stage 3 Extract data from the Employment file	
Input files	paye_employment [temp folder]

	paye_payslips <i>[temp_folder]</i> pidens <i>[temp_folder]</i>
Output files	employment_temp_combifile <i>[temp_folder]</i>
Structure of the output file	Temporary files indexed by HMRC_id and tax year
existing vars in the output files	all variables in the employment file
New variables created in the output file	None

This code collapses on the Employment data from multiple jobs, to give a single record for an individual and a tax year.

It also checks to see if the individual has no paid work in the employment in that year (by checking to see if payslips exist in that year). There are a small but notable number of individuals who have an employment record but no pay in a year.

Stage 4 Combine files into the ASHE supplement	
Input files	[*]_combi_file, described above <i>[temp_folder]</i> paye_pensioner_status <i>[temp_folder]</i> ashe_age_sex_lookup <i>[temp_folder]</i> pidens <i>[temp_folder]</i>
Output files	paye_ashe_supplement <i>[clean_folder]</i>
Structure of the output file	One file indexed by HMRC_id and tax year
existing vars in the output files	hmrc_id, tax_year
New variables created in the output file	variables as described above

The temporary files are brought together. The code also adds a marker for 'whether ever in ASHE' and 'whether in ASHE in the tax year'. ASHE piden is added and a variable 'year' (defined tax_year-1) is added to allow matching to ASHE on a year-by-year basis (ASHE is measured in April but referred to be calendar years, which are 1 less than the tax years recorded in the HMRC data).